รายงาน

การใช้คำสั่ง BLE เพื่อเปรียบเทียบค่าตัวแปร

โดย

นางสาววีรยา บุญแต่ง รหัสนักศึกษา 650910369

นางสาวภันธุ์ทิพา ปานคงคา รหัสนักศึกษา 650911022

เสนอ

ผศ.ดร. ยุทธนา เจวจินดา

รายวิชา 618242-65 สถาปัตยกรรมละองค์ประกอบคอมพิวเตอร์

ภาคเรียนที่ 2 ปีการศึกษา 2566

ภาควิชาวิศวกรรมไฟฟ้า คณะวิศวกรรมศาสตร์และเทคโนโลยีอุตสาหกรรม

มหาวิทยาลัยศิลปากร

จุดประสงค์ของโค้ด

โค้ดนี้ใช้ในการช่วยศึกษาการทำงานของ BLE(Brach Less or Equal) ซึ่งเป็นคำสั่งที่ใช้เปรียบเทียบสำหรับตัวแปรที่มีเครื่องหมาย โดยใช้เครื่องหมายน้อยกว่าหรือเท่ากับ และคำสั่งอ่านหน่วยความจำ

คำสั่ง branch

เป็นคำสั่งที่จะทำงานในตำแหน่งที่เราทำการระบุไว้ เมื่อเงื่อนไขเป็นจริง แต่หากคำสั่งไม่เป็นจริง ก็จะทำคำสั่งที่อยู่ในตำแหน่งถัดไป เทียบได้กับงานใช้คำสั่ง if/else ในภาษา c นั่นเอง โดยในที่นี้จะเน้นการใช้งาน BLE เป็นหลัก

Assembly code		C code
LDR R2, =G	; R2 = &G	
LDR R0, [R2]	; R0 = G	If (G < 7) {
CMP R0, #7		GLess7();
BGE next3	; if R0 >= 7	}
B GLess7		
Next3		

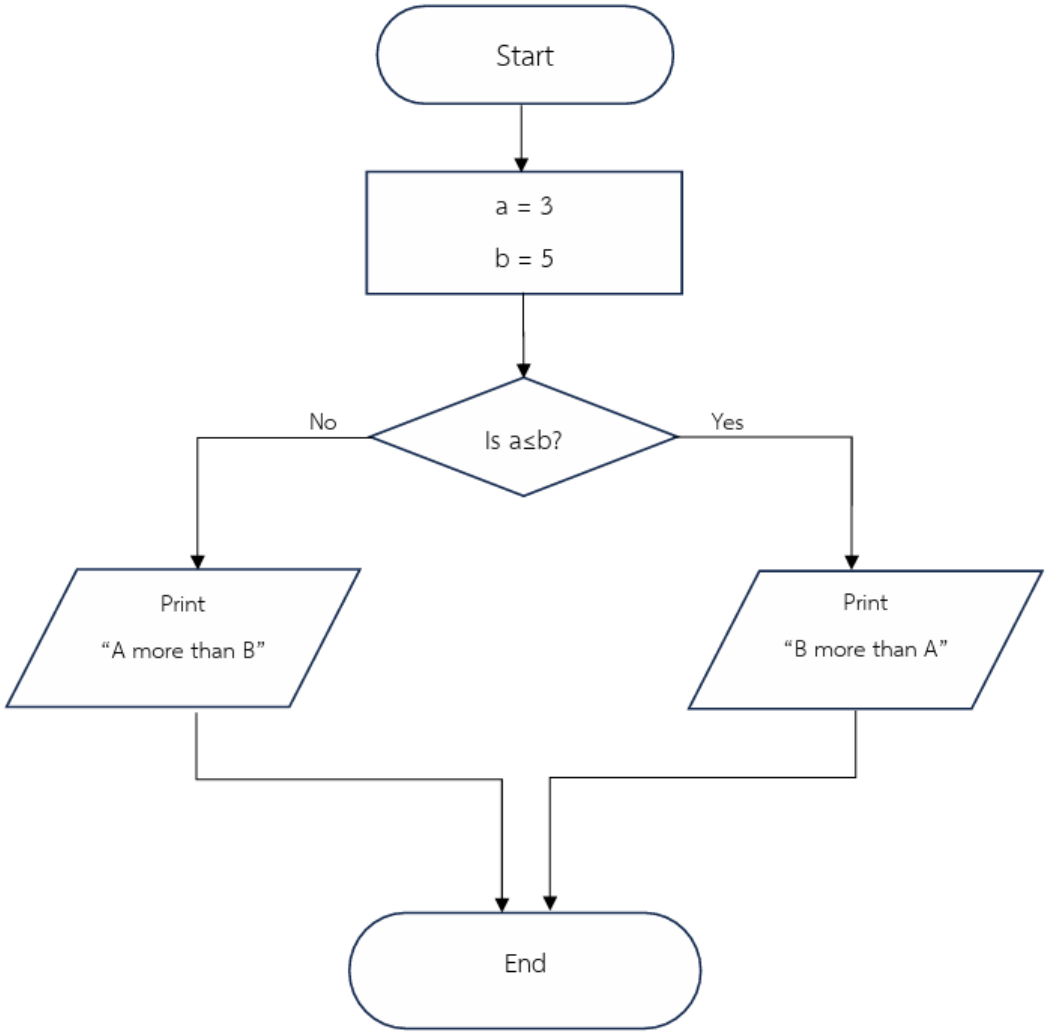
ภาพตัวอย่างการเทียบเปรียบโค้ดภาษา assembly กับ ภาษา C

การแสดงผลชุดตัวอักษร

ใช้คำสั่งติดต่อหน่วยความจำ ด้วยการเรียกใช้งาน syscall หมายเลข 4 ของ Raspbian Linux โดยใช้ register ทั้งหมด 4 ตัว ได้แก่

- R0 ใช้เก็บหมายเลขเป้าหมายการแสดงผล (ในที่นี้ใช้หมายเลข 1 คือ จอภาพ)
- R1 เป็นตำแหน่งหน่วยความจำที่เก็บตัวแปร string (ชุดตัวอักษร 8 บิต) ที่ต้องการแสดงผล
- R2 เป็นจำนวน byte ที่ต้องการแสดง
- R7 รหัส System call ของ Raspbian Linux โดย syscall 4 เป็นหารเขียนฟังก์ชัน และ syscall 1 แสดงค่าใน R0

Flowchart การทำงานของโปรแกรม



การทำงานของโปรแกรม

```
.global _start
_start:
        LDR R3,=a
        LDR R4,=b
        LDR R1,[R3]
        LDR R2,[R4]
```

ใช้คำสั่ง LDR ให้ R3 ชี้ไปที่ a และ R4 ชี้ที่ b

R1 ,R2 เก็บค่าตัวแปร a และ b ที่กำหนดไว้(a=3 ,b=5)

```
        CMP R1,R2
        BLE lessthan
```

ใช้คำสั่ง CMP และ BLE เปรียบเทียบตัวแปร R1,R2 โดยหาก R1≤R2 โปรแกรมจะข้าไปทำงานในส่วนของ lessthan แต่ถ้า R1>R2 โปรแกรมจะทำงานต่อในบรรทัดถัดไป

```
morethan:
        MOV R5,#0x41
        MOV R6,#0x42
```

ส่วน morethan ทำงานหาก R1 มีค่ามากกว่า R2 : กำหนดให้ R5 ,R6 เป็น A และ B ตามลำดับ

```
        LDR R1,=string
```

นำตำแหน่งหน่วยความจำของตัวแปร string มาเก็บไว้ใน R1

```
        STRB R5,[R1,#0]
        STRB R6,[R1,#12]
```

จากนั้นใช้คำสั่ง STRB เขียน R5 ที่มีค่าเป็น A ลงในตำแหน่งแรกของ string และเขียน R6 ที่เป็น B ลงตำแหน่งที่ 13 ของ string การทำงานในขั้นตอนนี้จะทำให้ค่าของ string ที่เป็น “X more than X\n” เปลี่ยนเป็น “A more than B\n”

```
        MOV R0,#1
        MOV R7,#4
        MOV R2,#14
        LDR R1,=string
        SWI 0
        b exit
```

ใช้ #1 เพื่อแสดงผลที่จอภาพ และ #4(syscall4) เพื่อเขียนฟังก์ชัน ใช้ R2 จองพื้นที่ให้มีค่าเท่ากับความยาวของข้อความ นำตำแหน่งของ string มาเก็บใน R1 อีกครั้ง จากนั้นข้ามไปส่วน exit

```
lessthan:
        MOV R5,#0x41
        MOV R6,#0x42
```

ส่วนของ lessthan ที่จะทำงานเมื่อ R1≤R2 : กำหนดให้ R5,R6 เป็น A และ B ตามลำดับ

```
        LDR R1,=string
```

นำตำแหน่งหน่วยความจำของตัวแปร string มาเก็บไว้ใน R1

```
        STRB R6,[R1,#0]
        STRB R5,[R1,#12]
```

จากนั้นใช้คำสั่ง STRB เขียน R6 ที่มีค่าเป็น B ลงในตำแหน่งแรกของ string และเขียน R5 ที่เป็น A ลงตำแหน่งที่ 13 ของ string การทำงานในขั้นตอนนี้จะทำให้ค่าของ string ที่เป็น “X more than X\n” เปลี่ยนเป็น “B more than A\n”

```
        MOV R0,#1
        MOV R7,#4
        MOV R2,#14
        LDR R1,=string
        SWI 0
```

ใช้ #1 เพื่อแสดงผลที่จอภาพ และ #4(syscall4) เพื่อเขียนฟังก์ชัน ใช้ R2 จองพื้นที่ให้มีค่าเท่ากับความยาวของข้อความ นำตำแหน่งของ string มาเก็บใน R1 อีกครั้ง

```
exit:
        MOV R7,#1
        SWI 0
```

เรียก syscall 1 เพื่อแสดงค่าใน Ro

```
.data
a:
 .byte 3
b:
 .byte 5
```

ส่วนข้อมูล : สร้างตัวแปร a เก็บค่า 3 และตัวแปร b เก็บค่า 5

```
string:
 .ascii "X more than X\n"
```

และประกาศตัวแปรชนิด string ให้เป็น “X more than X\n”

ผลการรัน :

```
pi@raspberrypi ~ $ as -o final2.o final2.s
pi@raspberrypi ~ $ ld -o final2 final2.o
pi@raspberrypi ~ $ ./final2
B more than A
```

คิวอาโค้ดสำหรับผลการรันโค้ดโปรแกรม